

## PRE-CLASS READING 1 OF 5

## Emerging Architectures for LLM Applications

Bornstein, M. & Radovanovic, R. (2023). Andreessen Horowitz (a16z). Published 20 June 2023.

### ASSIGNED READING

Full post — the reference architecture + the in-context-learning workflow ~15 min

Skim the two stack tables (layers + tools) — you don't need to memorise vendors ~5 min

» ~15 min (full post)

**Note on currency.** This is a 2023 snapshot. The specific tools churn fast — LangChain versions, whether Pinecone still dominates — but the three-stage workflow and the layer taxonomy are still exactly how LLM apps are assembled in 2026. Read it for the layers, not the logos.

### KEY TAKEAWAY

Most LLM applications are not built by training a model — they are assembled from a stack of off-the-shelf layers around a pattern called in-context learning. The dominant version is RAG (retrieval-augmented generation): embed your private data into a vector database, retrieve the most relevant chunks at query time, and paste them into the prompt. This turns an AI problem into a data-engineering problem most teams already know how to solve.

### WHY IT MATTERS FOR THIS COURSE

This is the blueprint for the whole of Class 2. Capsule C1 walks these exact layers (compute → data/embeddings → vector store → model → orchestration → serving → evals); C2 turns each layer into an own-vs-rent decision; C3 expands the three-stage in-context-learning workflow — which you then build in the RAG Colab, in two versions (a token-free local pipeline and an API one) so you feel the model-layer trade-off first-hand. In Class 1 you placed a single foundation-model API call next to a scikit-learn classifier; here that one call becomes an entire application built around the model.

**Feeds Class 2:** C1 (map the stack) • C2 (build-vs-buy) • C3 (in-context learning / RAG) • the RAG Colab.

### MAIN POINTS

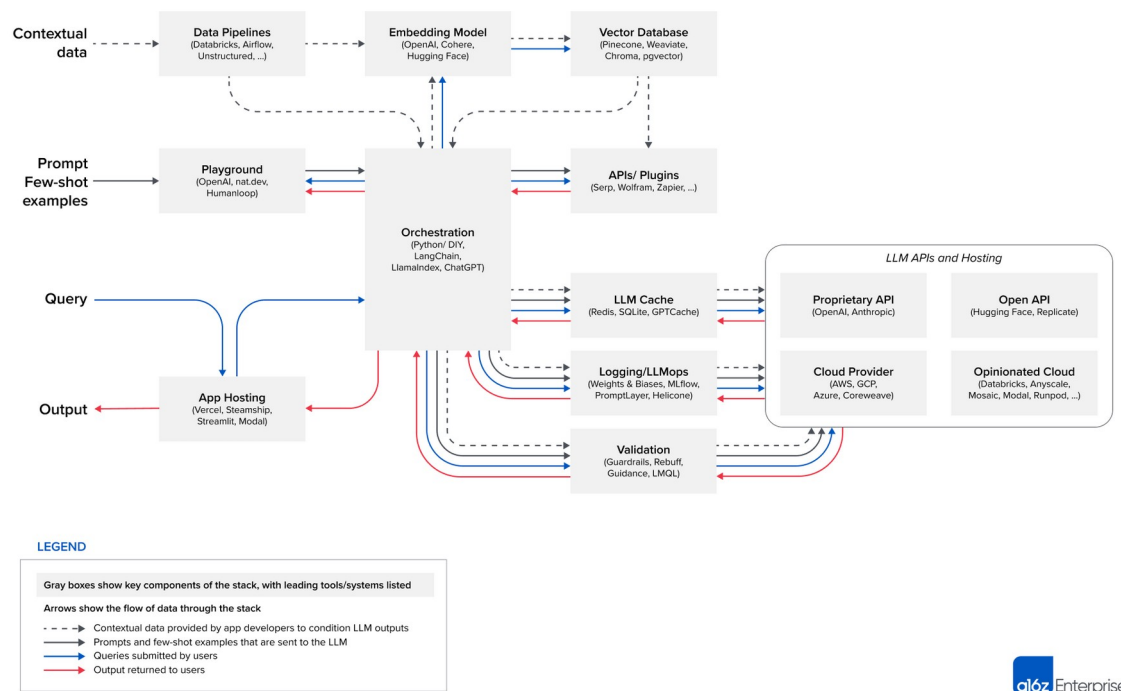
#### LLMs as a new building primitive

- **Three ways to build.** Train from scratch, fine-tune an open model, or call a hosted API. The stack in this post is built on the third — in-context learning — because that is where most teams start.
- **In-context learning.** Use an LLM off the shelf, with no fine-tuning, and control its behaviour by conditioning it on private 'contextual' data injected into the prompt.

#### The three-stage workflow (memorise this, not the tools)

- **1 • Preprocess / embed.** Break private documents into chunks, pass them through an embedding model, and store the resulting vectors in a vector database for later retrieval.
- **2 • Construct prompt / retrieve.** When a user asks a question, retrieve the most relevant chunks and assemble a prompt: a hard-coded template + few-shot examples + retrieved documents + any data from external APIs.
- **3 • Execute / infer.** Send the compiled prompt to a pre-trained model, then wrap it with operational systems — logging, caching, validation.
- **Why this beats fine-tuning for many apps.** No ML team or dedicated instance needed; it tends to outperform fine-tuning on small datasets (a fact must appear ~10+ times before fine-tuning 'remembers' it); and it can incorporate new data in near real time.

## Emerging LLM App Stack



The whole architecture on one page — the same layers this reading describes, and the layers we walk in capsule C1.

Source: Bornstein, M. & Radovanovic, R., "Emerging Architectures for LLM Applications," Andreessen Horowitz (a16z), 2023. Reproduced for classroom use.

### The layers of the stack

- **Data + embeddings.** ETL tools (Databricks, Airflow) load documents; an embedding model (OpenAI, Cohere, or open-source Sentence Transformers) turns them into vectors.
- **Vector database.** The systems-critical piece — stores and retrieves up to billions of embeddings (Pinecone, Weaviate, Chroma, pgvector).
- **Orchestration.** Frameworks (LangChain, LlamaIndex) assemble prompts, call APIs, retrieve context, and hold memory across calls.
- **Model APIs.** Proprietary (OpenAI, Anthropic) and open (Hugging Face, Replicate). Operational tooling adds caching (Redis), logging / evaluation (Weights & Biases, PromptLayer, Helicone) and validation (Guardrails, Rebuff).

### Build-vs-buy at the model layer

- **Start expensive, then optimise.** Teams often start on the strongest model, then trade down to a cheaper / faster one, try a second vendor, or triage some traffic to a fine-tuned open model for cost — a per-use-case build-vs-buy decision. This is exactly the choice the two RAG notebooks make concrete.
- **Agents are the missing layer.** In 2023 the authors flag agents as the big gap — powerful in principle (planning, tool use, memory) but 'they don't really work yet.' That is Class 4.

### READING FOCUS QUESTIONS — bring your answers to class

There are no wrong answers here — these surface how you think, not what you recall.

1. The post reduces 'build an AI app' to 'retrieve the right context and put it in the prompt.' Think of a task in your own work where the answer always depends on some private document or dataset. What would you need to retrieve — and how would you know you'd retrieved the right thing?
2. Fine-tuning changes how a model behaves; retrieval changes what it knows. For a use case you care about, name one thing you'd handle with retrieval and one you'd handle with fine-tuning — and explain why you wouldn't just fine-tune the facts in, even if they rarely change.
3. Every layer here is a build-or-buy choice. If you had to ship a prototype next week, which layers would you buy off the shelf and which — if any — would you build yourself, and why?

### SOURCE LINKS

Post • <https://a16z.com/emerging-architectures-for-llm-applications/>

Living component list • <https://github.com/a16z-infra/llm-app-stack>

Background — a16z 'AI Canon' • <https://a16z.com/ai-canon/>

## CAVEATS

- **1.** This summary is not a substitute for the reading. It is a map, not the territory. Read the source — it is the authority.
- **2.** This summary was generated and fact-checked using Anthropic models against the original a16z post. The architecture figure is a16z's own 'Emerging LLM App Stack' (Bornstein & Radovanovic, 2023), reproduced with attribution for classroom use. Tool names are point-in-time and will have moved. Flag anything that looks wrong.